

Anexo B — Tabla de compatibilidad de tipos

La tabla canónica que exige el enunciado está implementada en una **única** clase compartida: `com.navi.backend.semantic.TypeCompat` (apoyada por `com.navi.backend.semantic.TypeRules`). Los tres lenguajes (Lat, Y, Z) usan la misma tabla; los tipos concretos de cada lenguaje se mapean al sistema canónico `com.navi.backend.semantic.Type`.

B.1 Sistema de tipos canónico

TypeKind	Ejemplo de representación	Palabras en Lat	Palabras en Y	Palabras en Z
INT	int	numerus	entero	int
DOUBLE	double	decimalis	flotante	double
CHAR	char	littera	caracter	char
STRING	string	textum	cadena	String
BOOLEAN	boolean	bool	bool	boolean
VOID	void	ratio sin retorno / actio	definir sin ->	void
ARRAY	int[], int[][]	series	tipo id[n] / [] tipo	tipo[] / new tipo[n]
STRUCT	struct Nombre	(importado de .y)	estructura	—
CLASS	class Nombre	(importado de .z)	—	class
NULL	null	—	—	null
ERROR	(interno de recuperación)	—	—	—

Type es **immutable** y se compara por valor:

- ARRAY: mismo `elementType` y mismas `dimensions` (rango).
- STRUCT/CLASS: mismo nombre.
- Primitivos/void/error/null: por `kind`.

B.2 Ensanchamiento / promoción numérica

Regla: **CHAR** → **INT** → **DOUBLE**. Implementada en `TypeCompat.promoteNumeric(a, b)`.

Operando A	Operando B	Tipo promovido
char	char	char
char	int	int
char	double	double
int	char	int
int	int	int
int	double	double
double	char	double
double	int	double
double	double	double

Se considera "numérico" a char, int y double. boolean y string **no** son numéricos.

B.3 Asignación (`TypeCompat.canAssign(target, value)`)

Destino target	Origen value	¿Permitido?
cualquiera	mismo tipo exacto	Sí
class C	null	Sí
double	int, char, double	Sí
int	char, int	Sí
char	char	Sí
int	double	No (pérdida de precisión)
char	int, double	No

Destino target	Origen value	¿Permitido?
int/double	boolean/string	No
struct/class	agregado de otro nombre	No
int[]	int[] (mismo tipo y rango)	Sí

Importante: la asignación **no** permite degradar de double a int. Solo se permite ensanchar (char→int→double).

B.4 Operadores y tipo resultante (TypeRules)

Operador	Requisitos	Tipo resultante	Error si...
+ (suma/concat)	si algún operando es string → concatenación; si no, aritmética	string o el promovido numérico	ningún operando string y alguno no numérico
- * / %	ambos numéricos	promovido numérico	algún operando no numérico
- (unario)	operando numérico	el mismo tipo	operando no numérico
+ (unario, solo Z)	operando numérico	el mismo tipo	operando no numérico
!	operando boolean	boolean	operando no booleano
&&	ambos boolean	boolean	alguno no booleano
== !=	tipos comparables (ver B.5)	boolean	no comparables
< > <= >=	ambos numéricos	boolean	alguno no numérico
++ --	operando numérico	el mismo tipo	operando no numérico
+= -= *=	ambos numéricos	(sentencia/expresión Z)	alguno no numérico

Operador	Requisitos	Tipo resultante	Error si...
? : (ternario, Z)	ramas asignables al destino común	tipo de las ramas	ramas incompatibles

B.5 Comparaciones (TypeCompat.comparable(a, b))

Caso	¿Comparables?
mismo tipo exacto	Sí
class C vs null (en cualquier orden)	Sí
numéricos entre sí (char/int/double)	Sí
struct vs struct del mismo nombre	Sí (por equals)
string vs string	Sí (mismo tipo)
string vs int	No
struct A vs struct B	No
boolean vs int	No

B.6 Ejemplos concretos

Expresión	Resultado / Observación
"Edad: " + 20	string (concatenación; el entero se convierte a texto en runtime)
20 + 3.5	double
'A' + 1	int (el carácter se promueve)
'A' + 1.0	double
verdadero && falso	boolean
p1 == null (donde p1 : Persona)	boolean
p1.edad >= 18	boolean
x = 3.14 donde x : numerus	Error: no se puede asignar double a int

Expresión	Resultado / Observación
<code>int y = 'a'</code>	Permitido: char se ensancha a int
<code>"a" == 1</code>	Error: tipos no comparables

B.7 Arreglos

- El indexado conserva el rango: `int[][] → int[] → int` (`TypeRules.arrayElementType`).
- Cada nivel de un inicializador anidado consume una dimensión.
- En Y, los parámetros arreglo (`[] entero arr`) y estructura (`{ } E e`) se pasan **por referencia**; en Lat/Z los parámetros son por valor y los objetos Z son referencias (punteros en el heap).
- El índice debe ser numérico; indexar un no-arreglo es error.

B.8 Referencias de código

- `src/main/java/com/navi/backend/semantic/Type.java` — sistema canónico.
- `src/main/java/com/navi/backend/semantic/TypeCompat.java` — compatibilidad y promoción.
- `src/main/java/com/navi/backend/semantic/TypeRules.java` — validación de operadores, asignaciones, retornos y accesos.
- `src/main/java/com/navi/backend/semantic/{lat,y,z}/*TypeResolver.java` — mapeo de palabras clave a `Type`.